

Ascon on the World's Smallest Video Game Console

Brayden Connery

May 25, 2026

Abstract

In 2021, the Ohio based company TinyCircuits launched a Kickstarter campaign for the Thumbby: the world's smallest video game console [4]. The campaign concluded having raised over 13 times its original goal, and shipped out to backers in 2022. In August 2025, NIST standardized Ascon [10], a suite of cryptographic algorithms designed for use on "constrained devices", such as IoT devices and medical implants [3]. This paper describes how I implemented Ascon's symmetric encryption scheme "Ascon-AEAD128" on the Thumbby using C and C++ and how I tested the power efficiency of the algorithm.

Part I

Introduction

In 2021, TinyCircuits launched a Kickstarter campaign for the Thumbby: the world's smallest video game console [4]. It would go on to raise \$205,137 (over 13 times the initial goal of \$15,000), and be shipped out to backers in 2022. I thought the project looked neat, so, with the promise of it being "easily programmable" and having multiplayer functionality, I backed the project at a tier that got me two Thumbies, with the intention of programming some sort of game on them at a later date.

Later, in August 2025, NIST standardized Ascon [10], a suite of cryptographic algorithms designed for use on "constrained devices", such as IoT devices and medical implants [3]. NIST chose to standardize Ascon after it won the "CAESAR competition" which was held by the "international cryptologic research community" from 2012-2019 [2]. One feature of the standardized version of Ascon is its many modes of operations; The same basic "permutation function" can be used as a hash function, an extendable-output function, or, most importantly for this paper, an "Authenticated Encryption with Associated Data" (AEAD) cipher.

Finally, when I started taking a cryptography course in September 2025, I was surprised to see NIST's Ascon standard among the project options, given that it was only standardized a number of weeks before the course started! Given Ascon's focus on "constrained devices", I thought back to the Thumbies that I had sitting around since 2022 and figured it would be a great time to dust them off and start programming. Despite how many videos and articles I had seen that showcased how to program various microcontrollers, I had never actually programmed one myself. Furthermore, I had never implemented an algorithm *purely* based off its mathematical description either (I had previously relied on example

code and other implementations to aid my reading), so I figured this project would be a good way to tackle both problems.

For this report, I have implemented Ascon-AEAD128 using C, and created an accompanying Thumb-to-Thumb communication program using C++, in order to test the power efficiency of the algorithm when compared to sending plaintext messages.

Part II

Background

Before discussing the project itself, I will cover some important background information.

1 Ascon

The Ascon cipher was designed by Cristoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer in 2014 for submission to the CAESAR competition [5]. It went on to become the winner in the “Lightweight Applications” use case in February of 2019. Shortly after, NIST started working with the team to make Ascon into a standard, the first version of which was published in August 2025 [10].

The NIST-standardized version of Ascon has four modes of operations: Ascon-Hash256, Ascon-XOF128, Ascon-CXOF128, and Ascon-AEAD128. For this project I focused on the last of these modes: Ascon-AEAD128. AEAD stands for “Authenticated Encryption with Associated Data”, which is a specific type of symmetric key encryption scheme that not only lets Alice and Bob share messages securely, but also lets them authenticate that their messages haven’t been tampered with in transit. Future sections will elaborate more on how this works as it becomes relevant.

In the following sections, I have used a C-like pseudocode to help describe the inner workings of Ascon (and its AEAD algorithm). For more details, refer to the NIST document for a more math-focused description [10].

1.1 The Ascon Permutation Function

Ascon utilizes a “state” comprised of five 64-bit unsigned integers. The core permutation function (Algorithm 1) is defined as follows: All this method does is run the p_C , p_S , and p_L functions in sequence

Algorithm 1 Ascon-p[rnd]

Input: uint8 rnd, uint64[5] state

Output: None (state is modified in-place)

for i=0 to rnd-1 **do**

$p_C(state, rnd, i)$

$p_S(state)$

$p_L(state)$

end for

rnd times. Note that, as of version 1 of NIST’s standard, rnd must stay within $1 \leq rnd \leq 16$ [10], so space can be saved by using a small data type (such as a single byte) for this parameter.

1.1.1 Constant Addition Layer

The function p_C is called the “Constant Addition” layer (Algorithm 2), though computer scientists may prefer to call it the “Constant XOR” function, since it does not use modular integer addition.

Algorithm 2 p_C

Input: uint64[5] state, uint8 rnd, uint8 i

Output: None (state is modified in-place)

$state[2] = state[2] \oplus constTableFive[16 - rnd + i]$

This function’s purpose is to XOR index 2 of the state with the last rnd integers of *constTableFive* one at a time, once per round. In Ascon-AEAD128, only round values of 8 and 12 are used, so one *could* shorten or hardcode *constTableFive* if they desired. Furthermore, despite each entry in the table being 64-bits long, only the lowest 8-bits are used, giving further opportunities for compression. The values of *constTableFive* can be found in Table 5 of the NIST document [10].

1.1.2 Substitution Layer

Next, the function p_S is the “Substitution” layer (Algorithm 3), which uses an S-box to scramble each bit of the state column-wise. For the following function definition, let $\ll$ and $\gg$ denote the left and right shift operators respectively, $\wedge$ and $\vee$ denote bit-wise AND and OR respectively, and *my64BitInteger*[i] denotes accessing the i th bit of *my64BitInteger* (where 0 is the least significant and 63 is the most significant):

Algorithm 3 p_S

Input: uint64[5] state

Output: None (state is modified in-place)

for i=0 to 63 **do**

$index = (state[0][i] \ll 4) \vee (state[1][i] \ll 3) \vee (state[2][i] \ll 2) \vee (state[3][i] \ll 1) \vee (state[4][i])$

$sBoxOutput = SBoxTableSix[index]$

$state[4][i] = sBoxOutput[0]$

$state[3][i] = sBoxOutput[1]$

$state[2][i] = sBoxOutput[2]$

$state[1][i] = sBoxOutput[3]$

$state[0][i] = sBoxOutput[4]$

end for

Note that, in the process of extracting bits to be turned into an input for the S-Box, the bit order is *reversed* from the order of the integers in the state. For example, on the first loop, bit 0 in *state*[0] is placed in bit position 4 in the input *index*, while bit 0 in *state*[4] is placed in bit position 0.

The definition of *SBoxTableSix* can be found in Table 6 of the NIST document [10].

Alternatively, this function can be implemented by using the definition of the S-Box itself (Algorithm 4). Note that $\bar{x}$ denotes bitwise negation. This implementation may be preferred since it avoids a potentially

Algorithm 4 p_S

Input: uint64[5] state

Output: None (state is modified in-place)

```

state[0] = state[0]  $\oplus$  state[4]
state[4] = state[4]  $\oplus$  state[3]
state[2] = state[2]  $\oplus$  state[1]
s0 = state[0]  $\oplus$  (state[1]  $\wedge$  state[2])
s1 = state[1]  $\oplus$  (state[2]  $\wedge$  state[3])
s2 = state[2]  $\oplus$  (state[3]  $\wedge$  state[4])
s3 = state[3]  $\oplus$  (state[4]  $\wedge$  state[0])
s4 = state[4]  $\oplus$  (state[0]  $\wedge$  state[1])
s1 = s1  $\oplus$  s0
s0 = s0  $\oplus$  s4
s3 = s3  $\oplus$  s2
s2 =  $\bar{s}_2$ 
state[0] = s0
state[1] = s1
state[2] = s2
state[3] = s3
state[4] = s4

```

slow $O(64)$ loop in favor of a bunch of binary ANDs, XORs, and negations, which would be especially fast on hardware that supports 64-bit arithmetic.

1.1.3 Linear Diffusion Layer

Finally, p_L is the “Linear Diffusion” layer (Algorithm 5), which uses a bunch of rotates and XORs to scramble each entry of the state individually. With $\ggg$ representing the right rotation operator, the function is defined as follows:

Algorithm 5 p_L

Input: uint64[5] state

Output: None (state is modified in-place)

```

state[0] = state[0]  $\oplus$  (state[0]  $\ggg$  19)  $\oplus$  (state[0]  $\ggg$  28)
state[1] = state[1]  $\oplus$  (state[1]  $\ggg$  61)  $\oplus$  (state[1]  $\ggg$  39)
state[2] = state[2]  $\oplus$  (state[2]  $\ggg$  1)  $\oplus$  (state[2]  $\ggg$  6)
state[3] = state[3]  $\oplus$  (state[3]  $\ggg$  10)  $\oplus$  (state[3]  $\ggg$  17)
state[4] = state[4]  $\oplus$  (state[4]  $\ggg$  7)  $\oplus$  (state[4]  $\ggg$  41)

```

I have no idea how or why the authors picked those specific rotation values.

1.2 Ascon-AEAD128

The AEAD algorithm itself comprises of three main components: Encryption, Decryption, and some shared code for initialization and processing associated data.

1.2.1 Padding Helper

The padding algorithm (Algorithm 6) is used to handle a very common case: If the provided data for encryption/decryption is not a multiple of 16 in length, then padding will be required to properly read the associated data and plaintext.

Algorithm 6 Ascon-Pad

Input: uint64[5] state, uint8[] data

Output: None (state is modified in-place)

```
i = 0
while i < data.length do
    j =  $\lfloor i/8 \rfloor$ 
    s = i (mod 8)
    state[j] = state[j]  $\oplus$  (data[i]  $\ll$  (s * 8))
    i = i + 1
end while
j =  $\lfloor i/8 \rfloor$ 
s = i (mod 8)
state[j] = state[j]  $\oplus$  (0x01  $\ll$  (s * 8))
```

Essentially, the padding algorithm XORs as much data as is remaining into the state, then XORs a 0x01 into the the byte after. Note that this function is only valid when it is passed the last $0 \leq data.length \leq 15$ bytes of whatever data needs processing.

1.2.2 Shared Initialization/Associated Data Processing

Before we begin discussing the rest of the encryption algorithm, let *myArray*[*startInclusive* : *endExclusive*] denote the array slice from the element at *startInclusive* to the element at *endExclusive* - 1.

Now we can get to the definition of the shared algorithm (Algorithm 7) used for initializing the state and processing any associated data.

To start, the state is initialized with a constant/magic value, the given key, and the given nonce, then permuted. Afterwards, the state is XORed with the key once more, before beginning the associated data processing (if there is any).

The associated data processing step simply XORs the associated data with the state in blocks of 16 bytes at a time, then permutes after each XOR, before doing one last iteration to deal with remaining data. Note that this does *not* affect the associated data in any way, it simply modifies the state, which will then impact the “tag” produced at the end of encryption/decryption. To correctly use the algorithm, the associated data will be sent *unencrypted* over the channel, meaning it should only be used for non-sensitive data, such as communication protocol metadata or the nonce.

To cap off this shared process, a single 1 is XORed into the last (highest) bit of the state.

Algorithm 7 Ascon-AEAD128-Shared

Input: uint64[5] *state*, uint64[2] *key*, uint64[2] *nonce*, uint8[] *associated*

Output: None (*state* is modified in-place)

state[0] = 0x00001000808c0001 ▷ Initialization routine

state[1] = *key*[0]

state[2] = *key*[1]

state[3] = *nonce*[0]

state[4] = *nonce*[1]

Ascon-p[12](*state*)

state[3] = *state*[3] $\oplus$ *key*[0]

state[4] = *state*[4] $\oplus$ *key*[1]

if *associated.length* > 0 **then** ▷ Associated Data processing

i = 0

while *associated.length* - *i* $\geq$ 16 **do**

state[0] = *state*[0] $\oplus$ *associated*[*i* : *i* + 8]

state[1] = *state*[1] $\oplus$ *associated*[*i* + 8 : *i* + 16]

 Ascon-p[8](*state*)

i = *i* + 16

end while

 Ascon-Pad(*state*, *associated*[*i* : *associated.length*])

 Ascon-p[8](*state*)

end if

state[4] = *state*[4] $\oplus$ 0x8000000000000000

1.2.3 Encryption

Now for the definition of the encryption algorithm (Algorithm 8).

Algorithm 8 Ascon-AEAD128-Encrypt

Input: uint64[2] key, uint64[2] nonce, uint8[] associated, uint8[] plaintext, uint8[] ciphertext

Output: None (state is modified in-place)

uint64[5]state = {0, 0, 0, 0, 0}

Ascon-AEAD128-Shared(state, key, nonce, associated)

i = 0

while plaintext.length - i ≥ 16 **do**

▸ Blocks of 16 bytes

state[0] = state[0] ⊕ plaintext[i : i + 8]

state[1] = state[1] ⊕ plaintext[i + 8 : i + 16]

ciphertext[i : i + 8] = state[0]

ciphertext[i + 8 : i + 16] = state[1]

Ascon-p[8](state)

i = i + 16

end while

Ascon-Pad(state, plaintext[i : plaintext.length])

▸ Straggler bytes

while i < plaintext.length **do**

j = i (mod 16)

k = j (mod 8)

ciphertext[i] = (state[⌊j/8⌋] ≫ k) ∧ 0xFF

i = i + 1

end while

▸ Finalization

state[2] = state[2] ⊕ key[0]

state[3] = state[3] ⊕ key[1]

Ascon-p[12](state)

tag[0] = (state[3] ⊕ key[0])

tag[1] = (state[4] ⊕ key[1])

It starts off by running the shared initialization code to set up the state and process any associated data. Next, it iterates over the plaintext in blocks of 16 bytes, XORing them into the state, then running the permutation function. Finally, any remaining bytes are XORed into the state, the key is XORed into the state, and the permutation function is run one last time. The “tag” is then created by XORing part of the state with the key. This tag is what provides the “authentication” part of AEAD, and should be sent along with the encrypted message when communicating with Bob.

1.2.4 Decryption

Finally, we can discuss the definition of the decryption algorithm (Algorithm 9).

Like the encryption algorithm, the decryption algorithm also starts by running the shared initialization code, but it differs in the ciphertext processing loop: instead of simply XORing into the state, part of the ciphertext is XORed with part of the state to recover part of the plaintext, *then* part of the state is overwritten with part of the ciphertext. Other than that, the way data is ingested into the state is identical between encryption and decryption.

Algorithm 9 Ascon-AEAD128-Decrypt

Input: uint64[2] key, uint64[2] nonce, uint8[] associated, uint8[] ciphertext, uint8[] plaintext

Output: boolean (state is modified in-place)

uint64[5] state = {0, 0, 0, 0, 0}

Ascon-AEAD128-Shared(state, key, nonce, associated)

i = 0

while ciphertext.length - i ≥ 16 **do**

▸ Blocks of 16 bytes

 c₁ = ciphertext[i : i + 8]

 c₂ = ciphertext[i + 8 : i + 16]

 plaintext[i : i + 8] = state[0] ⊕ c₁

 plaintext[i + 8 : i + 16] = state[1] ⊕ c₂

 state[0] = c₁

 state[1] = c₂

 Ascon-p[8](state)

 i = i + 16

end while

while i < ciphertext.length **do**

▸ Straggler bytes

 j = i (mod 16)

 k = j (mod 8)

 l = ⌊j/8⌋

 plaintext[i] = ((state[j] ≫ l) ∧ 0xFF) ⊕ ciphertext[i]

 state[j] = (state[j] ∧ (0xFF ≪ l)) ∨ (ciphertext[i] ≪ l)

end while

j = i (mod 16)

k = j (mod 8)

l = ⌊j/8⌋

state[j] = state[j] ⊕ (0x01 ≪ l)

state[2] = state[2] ⊕ key[0]

▸ Finalization

state[3] = state[3] ⊕ key[1]

Ascon-p[12](state)

t₀ = (state[3] ⊕ key[0])

t₁ = (state[4] ⊕ key[1])

return t₀ == tag[0] ∧ t₁ == tag[1]

To finish, the tag is calculated and compared against the expected tag. If the tag matches, then the message was decrypted correctly and can be trusted, otherwise, the NIST paper says the erroneous plaintext should not even be returned from this function [10]. Given the practicalities of implementing this algorithm in C, my implementation simply returns “false” and lets the caller decide whether to discard or attempt to read the potentially malicious message. Also, the official Ascon reference implementation makes special note to check the tag in $O(1)$ time [1], which, I believe, is done to prevent an adversary from performing a timing attack to learn how much of the tag is correct.

2 Thumby

The Thumby is the world’s smallest video game console, measuring only $30.5\text{mm} \times 17.8\text{mm} \times 7.6\text{mm}$. It was created by the Ohio based company “TinyCircuits” and is powered by the RP2040, a 32-bit, dual-core, ARM M0+ based CPU designed by the Raspberry Pi foundation for use with the Raspberry Pi Pico [6]. The Thumby was announced via a Kickstarter campaign in September 2021 with the tagline “Ridiculously Tiny, Incredibly Cute, Easily Programmable” [4].

2.1 Available Programming Languages

In furtherance of it being an “easily programmable” *video game* console, TinyCircuits’ main focus regarding programming the Thumby has been to encourage new users to program their first game *ever* using MicroPython, a version of the Python programming language adapted for use on microcontrollers. The Thumby comes pre-loaded with an operating system that makes it easy to add/remove games written in MicroPython to/from the device, and they have extensive documentation and tutorials on how to use their custom Python API.

In 2023, Discord user “hemlockmay” and John van Leeuwen ported Google’s block-programming language “Blockly” to the Thumby [11], which takes inspiration from MIT’s “Scratch” programming language. This support is actually implemented by mapping the Thumby’s MicroPython API to Blockly’s block-based system, so one can easily switch between the MicroPython and Blockly representation of the code they’ve written.

The final way TinyCircuits offers to program the Thumby is using the Arduino IDE to program in C++. Unlike MicroPython and Blockly, there are no in-depth tutorials on how to program using this method, and the documentation is rather sparse in general, but it is certainly the fastest and most efficient option language of the three.

Part III

Implementation

In this section I will describe how I actually implemented Ascon and benchmarked it on the Thumby.

3 Choosing A Programming Language

From the three languages available on the Thumby, I chose to use a mix of C and C++.

While I do like Python as a programming language, I thought the inherent overhead of using it would skew any power consumption measurements. Since Blockly simply compiles to MicroPython, it shares the exact same downside. Plus, the fact that it's a block-programming language makes the thought of implementing a cryptographic algorithm seem a little ridiculous. While the documentation for the Thumby's Arduino C++ API is sparse, I determined that the performance/predictability benefits would be worth the slight increase in difficulty.

4 Implementing Ascon

I started off by writing a platform-agnostic implementation of Ascon in C. Originally, I based all my code off of the NIST paper, but I needed to check my work against some online resources to make sure I had implemented Ascon-p (Algorithm 1) correctly, since the NIST paper is rather sparse on the details of that particular function. With that said, I still avoided looking at any other implementations of Ascon until *after* mine was (seemingly) able to encrypt and decrypt a message.

To help test whether my implementation was working, I wrote a stand-alone test program that simply checked if encrypting a message then decrypting the result gave the same text. Eventually, in the interest of ensuring correctness, I modified the test program to check that my code was outputting the same ciphertext and tag as the reference implementation provided by the Ascon team [1].

5 Programming The Thumby

Once my implementation of Ascon was working, I started work on writing the Thumby-specific code.

5.1 Environment Setup

Thankfully, the small amount of documentation on TinyCircuits' website *does* describe how to compile and upload your code to the Thumby [9].

First, they recommend downloading the official Arduino IDE. In hindsight, I probably should've looked more into an alternative I learned about called "PlatformIO", which lets you program Arduino IDE compatible devices using VSCode. While I appreciated the simplicity and ease of use of the Arduino IDE, the lack of polished advanced features really made me wish I was using VSCode, or some similarly feature-rich IDE.

Next, TinyCircuits instructs you to load in some libraries: One for the main Thumby API, a dependency for the Thumby's screen, and a package called `arduino-pico` by Earle F. Philhower, III, which lets the Arduino IDE upload and debug code on the RP2040 and RP2350 series of processors [7]. Unfortunately, I quickly found out that not all of these libraries play nice together. If you download the latest version of all three libraries, then hit upload, the Thumby will have a completely black screen and not appear to respond to any button presses. As it turns out, the last update to the Thumby API (and screen dependency) is from 2022, while the last update to `arduino-pico` (at time of writing) is November 2025. After a little bit of searching and testing, the last version of `arduino-pico` that let me upload

working code to the Thumby was version 2.2.2, which released in June 2022. This is unfortunate since it means that I didn't have access to any debugging tools or firmware improvements that may have been added in later versions of the library. I can only assume that this incompatibility has arisen due to TinyCircuits' focus on the MicroPython and Blockly methods of programming. That said, all of the above libraries are open source, so I'm sure that the Thumby libraries could easily be updated to work with the latest arduino-pico version, but I didn't have enough time to dig into that for this project.

5.2 Memory Misalignment

Once I got code compiling onto the Thumby, I quickly found that it was crashing upon running my implementation of Ascon-AEAD128. At first I thought this might have been because of Ascon's reliance on 64-bit integers, which are not directly supported on a 32-bit processor such as the RP2040. However, after a bunch of isolated testing and frantic Google searches, I determined that it was likely because of memory alignment.

As alluded to in Section 1.1, the most efficient way to implement Ascon is to treat the given plaintext and additional data as a sequence of 64-bit unsigned integers, which the RP2040 will treat as a longer sequence of 32-bit integers since it lacks direct 64-bit support. In practice, this means allocating an array of bytes to store the ASCII characters of the message, then passing the address of that array in memory to the encryption/decryption functions, which will then cast it to the correct data type as needed. However, when allocating memory, the compiler will (usually) only allocate static arrays at multiples of the data size in bytes. For example, an array of bytes can go anywhere in memory, since $1 \mid x$ is always true, but an array of 32-bit integers will only be allocated on an address such that $4 \mid x$, since 32-bits is four bytes. This leads to the crux of the issue: The Cortex M0+ that the RP2040 is based on will crash if it is ever instructed to read/write n bytes at address a when $n \nmid a$.

So, since the data being allocated for the plaintext/ciphertext was an array of bytes (which can be allocated at any address), but the Ascon encryption/decryption functions expect to be able to treat that data as an array of 64-bit integers for ease of computation, the CPU may be instructed to read/write data that is misaligned, which causes a crash! To solve this, I made liberal use of the `alignas(n)` keyword, which is a C++ feature that lets you align arrays (and other data types) to any n you would like, as long as n is a multiple of the array's type's default alignment. In my case, this meant spamming `alignas(4)` on any and all arrays that the encryption/decryption functions would need to access.

6 Tester Design

Once I was able to successfully compile Ascon and have it run on the Thumby, I got to work designing how I wanted to test the AEAD algorithm's power usage.

6.1 Communication Protocol

In order to get an accurate estimate of the power draw of the algorithm in a real application, I was hoping to use the Thumby Link Cable, a custom double-ended micro USB cable, to make two Thumbies constantly encrypt, send, and decrypt messages back-and-forth as fast as they could. Unfortunately, I quickly realized that I had overestimated the Thumby Link Cable's capabilities; the cable is only half-duplex, not full duplex, so only one Thumby can be sending data at a time. I believe this stems

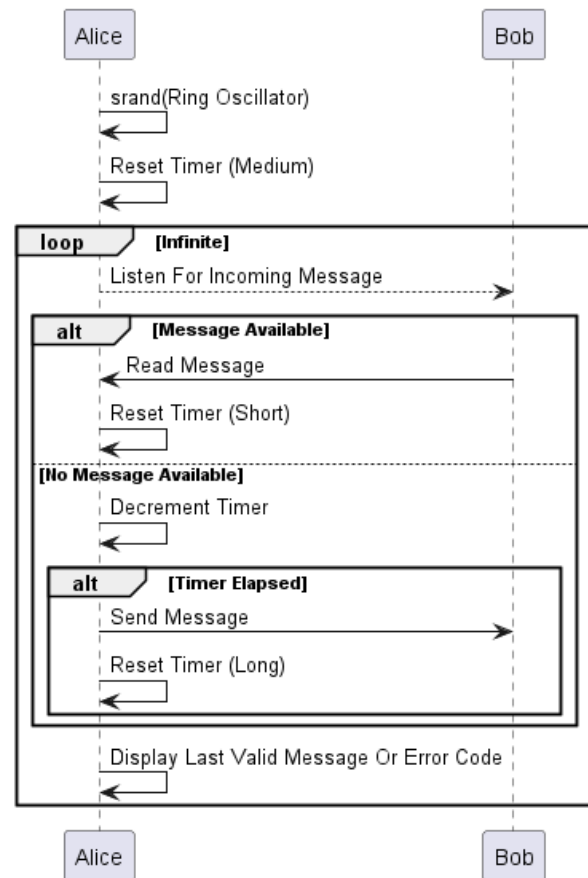
from the fact that the Thumbby Link Cable is not a standard “On The Go” USB cable [8], it actually has two wires flipped to enable communication in the first place.

Regardless, this meant I needed to come up with a communication protocol for the Thumbbies to use. I briefly considered implementing a common pre-established algorithm, such as Ethernet’s “CSMA/CD”, but instead I ended up making my own.

On boot, each Thumbby starts by reading 32 pseudo-random bits from the RP2040’s built in “Ring Oscillator”, then passes those bits as the seed standard C `srand()` function. This is done not for any cryptographically important reason, it is purely so that two Thumbbies started at the same time will not have the same seed. This fact is important because the documentation explicitly states that reading random bits from the Ring Oscillator “does not meet the requirements of randomness for security systems” [6].

Now that `srand()` has been seeded, the main communication protocol can begin. Essentially, the protocol uses the standard C `rand()` function to add a pseudo-random delay before every transmission, so that each Thumbby has ample time to listen for incoming messages in-between sending their own. I chose this system because it seemed to minimize delay time between messages while avoiding any overhead from more complicated algorithms, and avoiding any clock-sync issues that could have arisen from poor time-division multiplexing.

The exact method by which a new random timer value is chosen is a little bit janky: Each Thumbby keeps a 32-bit buffer filled with random bits from the standard C `rand()` function, then, whenever a new timer value is needed, takes b bits from that buffer, shifts them s bits left, and sets that as the new timer value. This system provides a basic way to get pseudo-random integers within a certain range without doing any division or rejection resampling (beyond checking that the random number retrieved wasn’t zero). When the Thumbbies first boot up they initialize the timer using $n = 4$ and $s = 6$, when resetting the timer after a message is received they use $n = 4$, $s = 4$, and when resetting after a message is sent they use $n = 4$, $s = 4$, but also forcefully add $2^{11} = 2048$ to the resulting value to ensure a minimum waiting time. Importantly, these timer values are not measuring any particular unit of time, they simply cause the Thumbby to avoid sending any messages for that many loop iterations, so the true time delay is based on the clock speed and speed of the encryption algorithm. The numbers I settled on are just educated guesses based on how fast the timer was counting down when I made the Thumbby display the timer on the screen, combined with being conservative about how long it would take to encrypt/decrypt messages using Ascon.



The communication protocol

Finally, note that the above algorithm is still missing lots of key components that would be needed in a real secure system. For example, since this project was not about secure key exchange, both Thumbies use a hardcoded key. Perhaps more importantly, while both Thumbies have a nonce that they use to encrypt and send to the other Thumbby, they never actually increment it. This was primarily done because I was worried about the performance implications of trying to replace the nonce every iteration, which may have distracted from the power consumption of the Ascon algorithm itself. One could argue, however, that any performance penalty incurred by such an operation *should* be included in the power consumption estimates, since a real system *would* have to do that to be secure.

6.1.1 Packet Format

With the protocol mostly complete, I then focused on addressing what data the Thumbies would actually be sending.

The Thumbby Link Cable is set to run at 115200 baud using the standard Arduino `Serial1` class (the `Serial` class is used for Thumbby to PC communication, which I used a tiny bit for debugging). In order to get as much throughput as possible out of this limited baud rate, I attempted to implement a smart pseudo-packet system to the Thumbby's communication protocol. This old protocol consisted of three fields: the total message length (2 bytes), the ciphertext message length (2 bytes), the nonce (16 bytes), and the actual data (ciphertext, tag, and associated data). The message was sent using multiple calls to `Serial1.write()` to send the packet: one for the all the length values and the nonce itself, one for the encrypted message, then one for the associated data. Furthermore, since each call could target a separate memory address, each Thumbby only needed to store one copy of the plaintext, ciphertext, and additional data. Unfortunately, while this system made great use of the available bandwidth by not sending (or storing) any more data than necessary, it turned out to be far too unreliable for long term communication.

Instead, in favor of getting the project done on time, I switched to using a fixed-size packet format. The maximum amount of data that the Thumbby supports sending in one call to `Serial1.write()` is 512 bytes, so, with the new system, each Thumbby maintains a 512 byte buffer that is used for sending and receiving messages. Note that separate, 4-byte-aligned buffers still need to be allocated for usage by the Ascon encryption and decryption functions, since this 512 byte buffer is 1-byte aligned. This change to the packet format made the "total message length" field from the old format redundant, so that field was replaced with "ciphertext length" instead. When sending, the data from the additional data buffer is copied first, then the encrypted ciphertext and tag are copied right after, before sending the data and clearing all buffers. When receiving, the received data is written into the buffer, before being copied into separate 4-byte-aligned buffers for decryption and display to the user.

Unfortunately, this new system poses a hard limit on message length: there are 24 bytes of overhead on every message, meaning the user can only send a total of $512 - 24 = 488$ bytes of data shared between the ciphertext and associated data, unlike the previous system which let the user send 512 byte ciphertexts and 512 byte associated data in the same message. Furthermore, this new system requires a whole 512 more bytes of memory! However, the new system does solve one important problem: Since each Thumbby is always reading the same amount of data (512 bytes), the entire packet header can be authenticated along with the associated data, whereas with the previous system the "total message length" field was impossible to authenticate, since the receiver would have no way to authenticate it without trusting it enough to read the rest of the packet. Now that both parties have agreed ahead of time on a fixed packet size, there is no way for a malicious party to try and trick Bob

into reading too much data; either the single call to `Serial1.read()` works, or it doesn't, there is no in-between. Plus, this new system actually works reliably, which is pretty important I guess.

6.2 User Interface

Given that the Thumby only has a D-pad and two buttons to work with, there were limited options for designing an effective text editor, but I'm pretty happy with what I came up with. Pressing the "B" button lets the user cycle between editing/viewing one of four fields: the outgoing message, the outgoing associated data, the incoming message, and the incoming associated data. The first two fields are editable by the user, while the last two are read only. Using left and right on the D-pad will change which character in the field is selected. For messages longer than 12 characters (the maximum number the Thumby can display on one line), the user can continue hitting left/right to scroll which part of the message is shown. Also, note that the character *after* the last *visible* character in the message is a valid selection. Next, pressing the "A" button will cause the currently selected letter to change its "group". There are four valid groups: lowercase letter, uppercase letter, digits, an underscore (to simulate a space), or null (to denote end of message). That last field is the reason why the user is allowed to select the character *after* the last visible one, since that's how they can add/remove letters from the message. Note there is a check to stop the user from writing outside of the allocated message buffer. Finally, pressing up and down on the D-pad will cycle the selected letter forwards and backwards through its group, wrapping around at either end (pressing down from 0 will cycle to 9, pressing up from Z will cycle to A, etc.).

I considered adding more functionality to the user interface, such as being able to choose which encryption algorithm to run, or even having a more complicated display to let the user switch between fields, but I settled on this simple system for a couple of reasons: First, it was relatively easy to code, since there is only one logical "screen" with no sub-menus. Second, and probably more important, it made testing more accurate, since there is no delay between booting up the Thumby and having it start running the communication protocol it was compiled to use.

7 Testing

Finally, with all the code written, it was time to test the code to see how long the battery lasted.

7.1 Methodology

As far as I can tell, the Thumby does not have any API for checking its battery level, but it does have an orange LED that will stay on when charging and appears to go off when full. As such, the first thing I did before each test was leave the Thumbies plugged in long enough for the LED to go out, then I configured the code to use the correct algorithm for the test and uploaded it to one of the Thumbies. Note that you can only upload code to the Thumby while it's on, so I made sure to turn it off *before* unplugging it after uploading the code, so as not to waste any battery before communication began. After repeating the upload process with the other Thumby, I plugged the Thumby Link Cable into both of them, then waited for my computer's clock to hit the top of the next minute. Once it did, I turned on both Thumbies and left them running. My original plan at this point was to note down the time I started them, then note down the time when I saw them power off due to low battery; However, I instead discovered that the Thumby has a very peculiar way of dealing with low battery. When a

Thumby starts to run low on battery it will slowly start powering off and on again. In hindsight, I estimate that they toggle state once every 5-10 seconds. As such, I decided to record the time when each Thumbby first started flickering individually. Finally, I calculated the difference between the start and end times to be the battery life under the given algorithm.

7.2 Results

It took many disorganized tests to finally settle on the above testing methodology, but once I finally had it settled, I used that methodology to collect as accurate of a test as possible for each configuration of the code and collected the data into this table:

Communication Method	Clear Flickering	Gray Flickering
None (Idle)	0:58	1:03
Plaintext	0:50	1:00
Encrypted with S-Box Lookup (Algorithm 3)	0:54	1:00
Encrypted with S-Box XOR (Algorithm 4)	0:51	1:00

The first column denotes which version of the algorithm was run in that test: “None” simply ran the user interface code and didn’t attempt to send or receive any messages, “Plaintext” used `memcpy()` to copy the input into the output buffer, while the two “Encrypted” rows ran their respective version of the encryption algorithm. The other two columns denote how long each Thumbby lasted from power on until they first started flickering (plus or minus around 10 seconds), the first being the time for the translucent-plastic Thumbby, while the second is for the solid gray Thumbby. While I performed many more than 4 tests, many of those early runs had inaccurate end timings, since I believe I looked down at the Thumbies after they had already been flickering for a while. As such, I only have four sample runs where I’m confident I accurately recorded the correct end times. Also, note that no consideration has been made to account for the potential battery degradation over the course of the failed tests, or over the course of testing the code in general.

By looking at the table, the main thing that jumps out to me is the stark difference in battery life between the clear and gray Thumbies. In the past, when I first played with the Thumbies, most of my time was spent using the gray one, so it is surprising that the clear one seems to have worse battery life overall! Does it have worse battery life because I let it sit without being played for longer, or was it like that from the factory? Furthermore, why is the gray Thumbby so consistent in lasting at least an hour regardless of the algorithm used?

I suppose one safe conclusion to draw is that *usage of the serial cable decreases battery life*, but that does little to explain why running an encryption algorithm led to longer battery life for the clear Thumbby. Perhaps when using plaintext more messages were being sent per second than when running an encryption algorithm, which caused the battery life to be drained faster? Though, the fact that the gray Thumbby didn’t seem to care pokes a bit of a hole in this idea.

Alternatively, maybe we should be concluding that *using Ascon has a negligible impact on battery life compared to sending plaintext*. Even given the limitations of these tests, no Thumbby user is going to notice if their Thumbby lasts 3-5 minutes less than normal. However, given all the other problems with this data, I am hesitant to truly use this as my final conclusion.

The most realistic conclusion is that *more data is needed before any conclusion on the power consumption of Ascon vs. plaintext can be made*. Ideally, many more samples should be taken and then analyzed

using statistical techniques, instead of the above table containing one accurate sample of each method. Perhaps the communication method should even be switched to time division multiplexing, just to counteract any potential influence from the speed at which messages are sent between devices.

8 Conclusion

I really enjoyed working on this project, though I am slightly baffled by the final results, and would definitely have done things differently given what I know now.

The process of turning Ascon from a mathematical description on the page into real, working code was a fun challenge, but I would definitely tackle it differently if I were to do it again. Instead of purely relying on the paper, I really should have compared against the reference implementation [1] to help clear up the confusion I had from reading the paper. Would this have “spoiled” some of the fun of writing all the code myself? Yes. Would it also have resulted in a working implementation of the algorithm much sooner? Also yes!

Programming in C/C++ on the Thumby was also fun, but I would never want to use this particular workflow again. While the simplicity of the Arduino IDE was convenient for getting started, it became far less convenient as the project grew in size. If I end up programming the Thumbies again, I will absolutely be looking into that “PlatformIO” project so I can use VSCode as my IDE of choice. I would also see about updating the Thumby libraries to work with the latest version of arduino-pico [7], and making a pull request so others could use the updated version. I think doing this would make C/C++ a much more appealing option for experienced devs wanting to get the most performance out of their Thumby.

Finally, had I known that testing Ascon’s power consumption on the Thumby would be so weirdly inconsistent, I may have gone to more extreme measures to get accurate readings. Perhaps popping the Thumby cases open and reading voltage drop across the battery terminals would be a more accurate way to measure power consumption, though I’m not sure if I could get the case back together again after doing that. Alternatively, maybe there’s some way for the RP2040 to measure its own power consumption, or for it to ask the battery controller to tell it how much is remaining? Failing both of those, simply allocating more time to test different communication protocols and take more samples would greatly improve the quality of data. Also, I would be curious to know how Ascon performs against a standard cipher such as AES128.

References

- [1] ascon, *ascon/ascon-c*, https://github.com/ascon/ascon-c/tree/main/crypto_aead/asconaead128/ref, 2025.
- [2] D. J. Bernstein, *Crypto competitions: CAESAR frequently asked questions*, <https://competitions.cr.yp.to/faq.html>.
- [3] Chad Boutin, *NIST finalizes ‘lightweight cryptography’ standard to protect small devices*, <https://www.nist.gov/news-events/news/2025/08/nist-finalizes-lightweight-cryptography-standard-protect-small-devices>, August 2025.
- [4] Ken Burns, *Thumby - the tiny playable keychain*, <https://www.kickstarter.com/projects/kenburns/thumby-the-tiny-playable-keychain>, September 2021.
- [5] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer, *Ascon v1*, Tech. report, Institute for Applied Information Processing and Communications, March 2014.
- [6] Raspberry Pi Ltd, *Rp2040 datasheet*, <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2025.
- [7] Earle F. Philhower, *earlephilhower/arduino-pico: Raspberry pi pico arduino core, for all rp2040 and rp2350 boards*, <https://github.com/earlephilhower/arduino-pico>, 2021.
- [8] TinyCircuits, *FAQ - by TinyCircuits*, <https://thumby.us/FAQ/>, 2023.
- [9] ———, *Setting up environment - by TinyCircuits*, <https://thumby.us/CCPP/Environment-Setup/>, 2023.
- [10] Meltem S  nmez Turan, Kerry McKay, Jinkeon Kang, John Kelsey, and Donghoon Chang, *Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions*, Tech. report, National Institute of Standards and Technology, August 2025.
- [11] Laver  na Wienclaw, *Blockly block-based programming on thumby*, <https://tinycircuits.com/blogs/news/blockly-block-based-programming-on-thumby>, March 2023.